

Applied AI Engineer

Case Study

Aug 2026

Part 1:

We provide no captures. Like the incumbent scanning apps, you own the problem from the phone's sensors onward. You choose one of two capture routes and both are legitimate:

Route 1: Your own iOS capture app. ARKit, RoomPlan, raw LiDAR depth, camera, IMU, whatever you pull from the SDK. Ship it as a TestFlight build or a dev build we can install on our device in under 10 minutes.

Route 2: A stock capture protocol. Name the off-the-shelf capture tool (a LiDAR logging app, the native camera, anything installable from the App Store) and write the one-page protocol a non-engineer follows: what to install, how to walk, how long, what to avoid, how to hand the files to your pipeline. At the defense we follow your page literally. If the page is ambiguous, the capture you get reflects that.

Three input tiers, all mandatory. The same output contract from each, with intervals that widen honestly as sensor data thins:

1. **Photos.** 2 to 8 stills per room from any iPhone 15 or newer, no depth, no poses. A set of photo folders, one folder per room, must produce the same stitched whole-property plan the other tiers produce, with intervals widened accordingly. This is the floor: any picture in, results out.
2. **Video.** A handheld walkthrough clip from any iPhone 15 or newer.
3. **LiDAR.** Depth, poses and intrinsics on Pro-class devices.

You submit a device matrix stating which tier runs on which hardware and what accuracy each tier honestly delivers.

Part 2: Output contract and gates

The full Round 1 contract per capture: dimensioned per-room plan with walls, ceiling height, floor area and openings, stitched multi-room plan with correct adjacency, per-surface damage regions with class and metric extent, concealed-damage flags with the rule that fired, scope line items keyed to surfaces, a confidence interval on every measurement, one command per capture, JSON to the published schema, rendered plan.

The stitched plan is the product surface: one whole-property floor plan a homeowner would recognise from poly.cam or magicplan, with every room placed, connected and dimensioned, produced from every tier including photos.

Because we provide no captures, you build the benchmark set yourself, and its composition is specified so it cannot be flattered:

- One multi-room capture, three or more rooms plus a connector
- One furnished room with staged damage spanning two damage classes
- The same rooms captured at all three input tiers, the multi-room set included: at the photo tier it arrives as per-room photo folders and must still stitch
- At least one room captured twice at the same tier, for the repeatability gate
- Laser or tape ground truth on everything, raw sensor data and measurements submitted

Round 1 gates apply, with five additions where the field collectively failed:

Metric	Gate
Opening widths	≤ 2 cm on $\geq 85\%$ of openings, detection itself scored: a missed opening and a phantom opening each count as a miss
Ceiling height	≤ 1.5 cm per room; where a room is captured more than once, spread across captures ≤ 1 cm. Repeatable-but-biased and unrepeatable both fail, and your report says which one you have
Repeatability	Two captures of the same room at the same tier agree within 1 cm or 0.5% per wall. Same room in, same plan out is what "spit out the same results" means operationally
Drift accountability	Your report states what you do about accumulated drift on the multi-room capture (loop closure, pose graph, plane-anchored correction, anything), and an ablation shows the stitched footprint with it on and off. "Poses used as-is" is an automatic fail on this row
Photo-tier whole-property stitch	Per-room photo folders produce one stitched plan with correct adjacency and no room overlaps; footprint within $\pm 8\%$ with calibrated intervals. A photo path that handles single rooms only fails this row

Photo-tier and video-tier gates are looser (photo: wall lengths within $\pm 8\%$ with calibrated intervals; video: $\pm 3\%$) but calibration is scored at every tier and confident garbage on thin input caps your total score.

Part 3:

On 2 of your benchmark rooms: your pipeline's output at the LiDAR tier against one consumer scanning app of your choice on the same rooms (free tier is sufficient; name the app and version, submit its export). One table, your error and theirs, dimension by dimension. Beat or tie on $\geq 70\%$ of shared dimensions.

Cost is not an accepted reason; free tiers exist.

Part 4: The fix loop (25% of score)

Acknowledge a one-page fix declaration:

1. The single worst-performing gate in your own benchmark, with the failing number
2. Your root-cause hypothesis and the evidence for it
3. The fix you intend to ship and the number you predict after it

Then you ship it. Final submission contains the before run, the after run, both regenerable by us, and a readable diff. Scoring:

- Correct root cause, shipped fix, gate moves from fail to pass: full marks
- Correct root cause, shipped fix, meaningful movement short of the gate: majority marks when the report says why it fell short
- Prediction badly wrong either direction: marks for the post-mortem's honesty, none for the prediction
- Analysis with no shipped fix: zero, regardless of quality
- Fix with no regenerable before/after: zero

Part 5: Process evidence

Commit as you work. We read the history. A repo that materialises fully formed in one or two commits at the deadline scores zero on this part and its narrative report is read with proportional suspicion. We are not counting commits per day; we are checking the history could belong to the person who built the thing.

AI coding tools remain fully allowed and expected. The defense is where that account settles: every design decision defended live, tools closed.

Deliverables

1. **Compliance matrix:** requirement → file path → artifact → status.
2. **Capture route:** the TestFlight/dev build, or the one-page stock-capture protocol. Plus the device matrix.
3. **Repo,** README to running on a fresh capture in under 15 minutes on a clean machine, one command per capture.
4. **Reproduction bundle:** everything our machine needs to regenerate every reported number from raw inputs. Cached model outputs acceptable when the cache replays deterministically and the live path also runs, because the walk-in test runs live.
5. **Benchmark report:** gates at all three tiers, repeatability table, head-to-head table, timing.
6. **Fix loop bundle**
7. **Technical report, max 6 pages.** Architecture, tier design and device matrix, drift handling, error budget, calibration analysis, the fix loop story, known failure modes. The page cap is deliberate: report length trades against engineering time, and we score the engineering.
8. **Raw benchmark data:** sensor logs, ground truth, app exports.

The walk-in test

At the defense we capture a space you have never seen with our own iPhone 15 or newer, choosing the tier on the day, following your capture route exactly. Your pipeline runs on it cold, in front of us. We measure the space with a laser measurer while it runs and score your output against those measurements on the spot. All three tiers must be ready to run.

Scoring

Weight	Component	Failure mode it exists to catch
30%	The walk-in test: cold run on our capture, scored against our laser measurements taken in the room	Systems that only work on their author's data
25%	Fix loop delta	Diagnosis shipped in place of repair
15%	Verified benchmark accuracy across all three tiers	Claims that evaporate under reproduction
10%	Compliance matrix coverage	Building a different product than the one specified
10%	Head-to-head vs incumbent app like magicplan or poly.cam	Benchmark avoidance
5%	Capture route quality: install time, protocol clarity, what a non-engineer would experience	Pipelines with no path to a user's hands
5%	Process evidence	Unauditable single-commit repos

Constraints

- Handheld consumer capture only, any pretrained model, dataset or API with disclosure, everything runs without calling your infrastructure. Same as Round 1.
- Weights and large binaries fetched by script or volume. Real properties contain mirrors, glass, wet-look surfaces and low light. Cover them in your submission.